

Pokémon Cameo Database from Pokémon Based List to Pokémon Card Based List

Design Doc: `cameo-convert` — Inverting the Cameo Pokémon Card Database

Status	Approved — decisions locked in §10
Author	<your name>
Reviewers	<TBD>
Last updated	2026-05-23
Related	RotomAmiti's Cameo Pokémon Card Database (<code>.ods</code> source)

1. Overview

The Cameo Pokémon Card Database is currently published as a single `.ods` file maintained by RotomAmiti. The file lists, for every Pokémon (and many Trainers), the cards they appear on **without being named on the card** — e.g. the Natu on Legendary Treasures *Bouffalant*, or Weedle on Base Set *Scoop Up*.

Today the lookup direction is:

```
1 <Pokémon Name>: [List of cards this Pokémon appears in]
2
```

This doc proposes `cameo-convert`, a Python package that produces the inverse view:

```
1 <Card>: [List of Pokémon that cameo on this card]
2
```

...plus a GitHub Actions workflow that runs the conversion on every published GitHub Release and attaches the generated files (in many formats, two variants each) as release assets.

2. Motivation

- Collectors today can answer "*which cards feature Pikachu as a cameo?*" but not "*which Pokémon cameo on this card I just pulled?*". The latter is the more common in-the-moment question.

- The source `.ods` is a single human-maintained spreadsheet. Every consumer currently has to either open it in LibreOffice or maintain their own parser. A standard set of derived files removes that friction.
- Publishing on every GitHub Release ties the derived data to a versioned, citable snapshot.

3. Goals & Non-Goals

Goals

- Read the `.ods` source from the repo and produce inverted (`card` → `[pokémon]`) datasets.
- Emit **two variants** of each output:
 - a. **Clean** — English-only collector view (see §6.2).
 - b. **Full** — every entry, with edge-case metadata preserved.
- Emit each variant in **13 formats** (see §6.3).
- Run automatically when a release is published (and shows as "Latest" on the repo sidebar) and upload all generated files as release assets, with the release tag in every filename.
- Be runnable locally as a CLI (`python -m cameo_convert ...`) for testing and ad-hoc use.

Non-Goals

- Re-authoring or curating the source data. The `.ods` is the source of truth; the package is a downstream transformer.
- Supporting input formats other than `.ods`.
- A web UI, search frontend, or hosted API. Consumers download the generated files.
- Backfilling historical releases — only releases from adoption forward.

4. Background: Source Data Anatomy

This section is the result of an audit of the current `.ods`. Implementers should treat it as canonical for v1.

4.1 Sheets

Sheet	Purpose	Rows (current)
Main	Human-facing documentation, legend, sources	~15
Gen 1 ... Gen 9	Pokémon cameo listings, one sheet per	~100–1,200 each

	generation	
Trainers	Human characters (cameo Trainers + Trainer cards)	~540

The **Main** sheet is metadata only and is **not** ingested for cameo data, but its legend (italics, blue background, merged cells) defines the meaning of the formatting in the other sheets.

4.2 Column layout

Generation sheets (Gen 1 ... Gen 9):

Col	Header	Notes
0	Ndex	National Pokédex number. Merged vertically across all rows belonging to one Pokémon.
1	Cameo Pokémon	Species name. Merged vertically alongside Ndex . May be a different form (e.g. "Mega Venusaur" listed under Venusaur's block).
2	Card name	The card on which the cameo appears. May be merged vertically to indicate multiple set #'s share the same artwork.
3	Set	The set the card belongs to.
4	#	Collector number. Not always numeric: - , GG61 , TG16 , 152b ,

		Parent-Child are all observed.
5	Notes	Free text. Common values: Jumbo , silhouette , picture , costume , Ditto's disguise , only partially visible , language tags.

Trainers sheet:

Col	Header	Notes
0	(region)	KANTO , JOHTO , etc. Merged across that region's Trainers. No header text in row 0.
1	Cameo Trainer	Character name. Merged vertically.
2	Card name	Same conventions as Gen sheets. Occasionally blank.
3	Set	Same.
4	#	Same.
5	Notes	Same.

Per the Main sheet's documentation, Trainers behave **inversely** to Pokémon:

Pokémon cameo entries list cards where the species is *not* named. Trainer entries list cards where the character *may or may not* be named — the rule for Trainers is "include any Pokémon card on which the character appears regardless of the name."

4.3 Formatting carries data

The `.ods` encodes meaning in cell formatting that the bare cell text does not capture:

Visual cue	Meaning	Detection
<i>Italic</i> card name	Visual edge case — Pokémon may be partly out of frame, a toy, a costume, a silhouette, etc. The <code>Notes</code> column elaborates.	Look up the cell's <code>style-name</code> → <code>style:text-properties[@fo:font-style="italic"]</code> in <code>content.xml</code> .
Light blue background (<code>#cfe2f3</code>)	Card was never released in English. If also not in Japanese, the language is named in <code>Notes</code> .	<code>style:table-cell-properties[@fo:background-color="#cfe2f3"]</code> .
Merged cells on cols 0–1	All rows in the merge belong to the same Pokémon / Trainer.	<code>@table:number-rows-spanned > 1</code> ; subsequent rows have <code><table:covered-table-cell></code> placeholders.
Merged cells on col 2 (<code>Card name</code>)	The merged set numbers all share the same artwork (a reprint group).	Same mechanism, scoped to col 2.

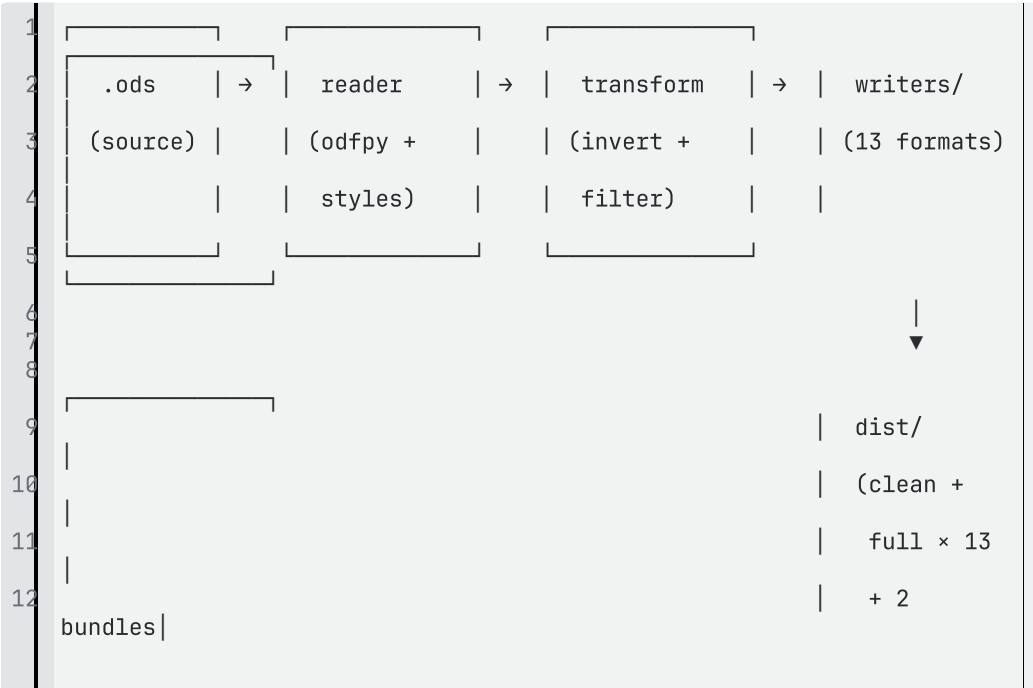
Implication: we cannot use `pandas.read_excel(engine="odf")` alone, because it discards formatting. We need direct XML/odfpy access.

4.4 Edge-case taxonomy (as observed)

Category	Signal	Example
----------	--------	---------

Not-in-frame / partial	italic + Notes: "only partially visible"	Togepi on Southern Islands #4 (under Ivysaur)
Represented by toy/silhouette/picture	italic + Notes: "silhouette" "picture" "costume"	Team Rocket's Meowth, Wizards Promos #18
Disguise (different species)	italic + Notes: "Ditto's disguise"	Ditto disguised, Delta Species #36
Jumbo card	Notes: "Jumbo"	Pokémon Valley promo
Non-English release	light blue background	Many XY-P / S-P / SWSH promos
Same artwork reprints	col 2 merged across rows	Multiple set #'s under one Card name
Cameo is a different form	Cameo Pokémon ≠ row's species	"Mega Venusaur" under Venusaur block
Blank card name	empty col 2	Some Trainers rows (e.g. Call of Legends #77 under Misty)

5. High-Level Design



```
13 _____
14
```

Stages are independent and testable in isolation. The reader's output is a plain-Python intermediate representation (§6.1); transform/filter operate on that IR; writers consume the final IR.

6. Detailed Design

6.1 Intermediate representation

```
1 @dataclass(frozen=True)
2 class CameoEntry:
3     cameo_subject: str          # "Mega Venusaur" – literal from
    spreadsheet
4     parent_species: str | None  # "Venusaur" when subject is a form;
    else None
5     subject_kind: Literal["pokemon", "trainer"]
6     ndex: int | None            # only for pokémon
7     region: str | None          # only for trainers ("KANTO"...)
8     card_name: str | None       # may be None (rare)
9     set_name: str
10    collector_number: str        # string – "152b", "-", "Parent-
    Child"
11    notes: str | None
12    flags: frozenset[Flag]
13    artwork_group_id: str | None # synthetic id grouping reprints w/
    shared art
14
15 class Flag(Enum):
16     ITALIC = "italic"           # visual edge case (Notes
    specifies)
17     NON_ENGLISH = "non_english" # light blue background
18     JUMBO = "jumbo"             # Notes contains "Jumbo"
19     BLANK_CARD_NAME = "blank_card"
20     DIFFERENT_FORM = "different_form" # cameo_subject ≠
    parent_species
21     SHARED_ARTWORK = "shared_artwork" # merged in Card name col
22
```

The inverted output groups by **card identity** = `(set_name, card_name, collector_number)`. Per D2 (no reprint collapsing), every collector number gets its own card entry, even when the source merged them under one Card name; entries with the same artwork share an `artwork_group_id` so consumers can collapse if they want.

6.2 Clean vs Full variant rules

Clean — English-only collector view. Excludes:

- `Flag.NON_ENGLISH` (light blue background — never released in English)
- `Flag.ITALIC` (visual edge case: toy, silhouette, costume, partial frame, disguise)
- `Flag.BLANK_CARD_NAME` (can't reference a card with no name)

Full — everything from the source, with every flag preserved as metadata so downstream consumers can re-filter however they want.

6.3 Output formats (13 total)

Each variant (Clean, Full) is emitted in every format below.

Format	Extension	Best for
JSON	<code>.json</code>	Programmatic consumption — pretty-printed, one top-level object keyed by card identity.
JSON Lines	<code>.jsonl</code>	Streaming / <code>jq</code> users — one card per line.
YAML	<code>.yaml</code>	Human-editable configs.
TOML	<code>.toml</code>	Static config consumers, Rust/Python tooling.
XML	<code>.xml</code>	Legacy tooling. Schema in README.
CSV	<code>.csv</code>	Spreadsheet import — long format, one (card, cameo) pair per row.
TSV	<code>.tsv</code>	Same shape; safer for cells containing commas.
Markdown	<code>.md</code>	Reading on GitHub — grouped by set, then card.
HTML	<code>.html</code>	Browse in browser — single self-contained

		page, searchable client-side.
Plain text	.txt	Lowest common denominator — Card (Set #): A, B, C per line.
SQLite	.sqlite	Local SQL querying — two tables: cards , cameos .
XLSX	.xlsx	Re-import to Excel — one row per (card, cameo).
ODS	.ods	Round-trip for LibreOffice users — same shape as XLSX.

Bundles: in addition to the individual files, the workflow uploads `cameo-convert-{tag}-all-formats.zip` and `.tar.gz` containing every output, so users can grab one archive.

Total assets per release = (13 formats × 2 variants) + 2 archives = **28 files**.

6.4 Output schema (canonical, for JSON)

```

1 {
2   "metadata": {
3     "source_file": "cameo-database.ods",
4     "source_last_updated": "2026-05-21",
5     "generated_at": "2026-05-23T12:00:00Z",
6     "generator_version": "1.0.0",
7     "release_tag": "v1.2.3",
8     "variant": "clean",
9     "total_cards": 4123,
10    "total_cameo_entries": 7891
11  },
12  "cards": {
13    "Aquapolis|Town Volunteers|136": {
14      "card_name": "Town Volunteers",
15      "set": "Aquapolis",
16      "collector_number": "136",
17      "artwork_group_id": null,
18      "cameos": [
19        {
20          "subject": "Bulbasaur",
21          "parent_species": null,
22          "kind": "pokemon",
23          "ndex": 1,
24          "notes": null,
25          "flags": []

```

```

26     }
27   ]
28 },
29 "Evolutions|Venusaur Spirit Link|89": {
30   "card_name": "Venusaur Spirit Link",
31   "set": "Evolutions",
32   "collector_number": "89",
33   "artwork_group_id": null,
34   "cameos": [
35     {
36       "subject": "Mega Venusaur",
37       "parent_species": "Venusaur",
38       "kind": "pokemon",
39       "index": 3,
40       "notes": null,
41       "flags": ["different_form"]
42     }
43   ]
44 }
45 }
46 }
47

```

On D7 ("both" for different-form cameos):

- **Structured formats** (JSON, YAML, TOML, XML, SQLite): one entry per source row, both `subject` (literal: "Mega Venusaur") and `parent_species` ("Venusaur") populated. The `different_form` flag marks the case.
- **Flat formats** (CSV, TSV, MD, TXT, XLSX, ODS): the row is emitted **twice** — once with the literal name in the searchable name column, once with the species name — so a naïve `Ctrl-F` for either "Mega Venusaur" or "Venusaur" finds the card. A `relation` column (`literal` / `species_parent`) distinguishes them.

6.5 Package layout

```

1  cameo-convert/
2  |— pyproject.toml
3  |— README.md
4  |— LICENSE
5  |— .github/
6  |   |— workflows/
7  |   |   |— release.yml
8  |— data/
9  |   |— cameo-database.ods          # source, normalized filename,
   |   |checked into repo
10 |— src/
11 |   |— cameo_convert/
12 |   |   |— __init__.py            # version, public re-exports
13 |   |   |— __main__.py           # `python -m cameo_convert`
14 |   |   |— cli.py                # argparse entry
15 |   |   |— model.py              # dataclasses, Flag enum
16 |   |   |— normalize.py          # string normalization flow
   |   |($6.8)
17 |   |— reader.py                 # ODS → IR (odfpy + style
   |   |resolution)
18 |   |— transform.py              # inversion + grouping
19 |   |— filter.py                 # clean vs full
20 |   |— writers/
21 |   |   |— __init__.py           # writer registry
22 |   |   |— base.py               # Writer protocol

```

```

23 |         |— json_writer.py
24 |         |— jsonl_writer.py
25 |         |— yaml_writer.py
26 |         |— toml_writer.py
27 |         |— xml_writer.py
28 |         |— csv_writer.py
29 |         |— tsv_writer.py
30 |         |— md_writer.py
31 |         |— html_writer.py
32 |         |— txt_writer.py
33 |         |— sqlite_writer.py
34 |         |— xlsx_writer.py
35 |         |— ods_writer.py
36 |     |— tests/
37 |         |— fixtures/
38 |             |— tiny_sample.ods          # 20-row hand-built fixture
39 |         |— test_reader.py
40 |         |— test_normalize.py
41 |         |— test_transform.py
42 |         |— test_filter.py
43 |         |— test_writers.py
44 |

```

Distribution / repo name is `cameo-convert` (hyphen); importable package is `cameo_convert` (underscore) — PEP 8 / PyPI convention.

Source file is renamed from `RotomAmiti's Cameo Pokémon Card Database.ods` to `cameo-database.ods` at commit time. The apostrophe and `é` in the original tripped CI shell quoting in scratch tests.

6.6 CLI

```

1 | python -m cameo_convert \
2 |     --input data/cameo-database.ods \
3 |     --output-dir dist/ \
4 |     --release-tag v1.2.3 \
5 |     --formats
6 |     json,jsonl,yaml,toml,xml,csv,tsv,md,html,txt,sqlite,xlsx,ods \
7 |     --variants clean,full \
8 |     --bundle zip,tar.gz

```

All flags have sane defaults. `--release-tag` defaults to empty (in which case filenames omit the tag segment) so local runs produce `cameo-convert-cards-clean.json` and CI runs produce `cameo-convert-v1.2.3-cards-clean.json`.

6.7 Dependencies

Package	Why
<code>odfpy</code>	ODS parsing with access to formatting (italics, bg color,

	merges).
<code>openpyxl</code>	XLSX writing.
<code>PyYAML</code>	YAML writing.
<code>tomli-w</code>	TOML writing (Python 3.11+ has <code>tomllib</code> for reading only).
<code>Jinja2</code>	HTML and Markdown templates.

Standard library covers JSON, JSONL, CSV, TSV, SQLite, XML, TXT, ODS writing. No pandas — overkill for this workload.

6.8 String normalization flow

Three normalization layers, in order of decreasing fidelity. Implement in `normalize.py` so consumers (and other projects) can replicate.

Layer 1 — `canonicalize(name)` (display form)

What you store and display. Loses nothing semantic.

1. Unicode NFC normalization (`unicodedata.normalize("NFC", s)`) — combines decomposed sequences (`e` + combining acute → `é`), so `"Pok\u00e9mon"` and `"Poke\u0301mon"` become byte-identical.
2. Replace typographic apostrophes with the ASCII straight apostrophe:
 - `\u2019` (right single quotation mark) → `'`
 - `\u2018` (left single quotation mark) → `'`
 - `\u02BC` (modifier letter apostrophe) → `'`
3. Collapse runs of whitespace to a single space, strip leading/trailing.

Input	Output
<code>Farfetch\u2019d</code>	<code>Farfetch'd</code>
<code>Sirfetch\u2019d</code>	<code>Sirfetch'd</code>
<code>Poke\u0301mon</code> (NFD)	<code>Pokémon</code>
<code>Mr. Mime</code> (double space)	<code>Mr. Mime</code>

Layer 2 — `match_key(name)` (lookup form)

For case- and accent-insensitive equality matching across messy sources. Lossy by design.

1. `canonicalize(name)`.
2. **Preprocess gender symbols** (*before stripping non-alphanumerics, or you lose Nidoran♀ vs Nidoran♂*):
 - `\u2640` (♀) → `-f`
 - `\u2642` (♂) → `-m`
3. `casefold()` — not `lower()`. Handles German ß → ss, Turkish dotless-i, etc.
4. NFKD normalization, then drop combining marks:

```
1 s = unicodedata.normalize("NFKD", s)s = "".join(c for c in s if not
2 unicodedata.combining(c))
```

This is what strips accents: `é` decomposes to `e` + combining-acute, and the combining-acute is filtered out.

5. Strip everything except `[a-z0-9]`.

Input	match_key
Farfetch'd	farfetchd
Flabébé	flabebe
Mr. Mime	mrmime
Ho-Oh	hooH
Type: Null	typenull
Iron Jugulis	ironjugulis
Tapu Koko	tapukoko
Porygon-Z	porygonz
Nidoran♀	nidoranf
Nidoran♂	nidoranm
Pokémon	pokemon

Edge case to know about: `match_key("Mime Jr.")` and `match_key("Mr. Mime")` both end up with `mimejr` / `mrmime` respectively — fine. But beware: `match_key("Tauros`

(Paldean Combat Breed)") becomes `taurospaldeancombatbreed`, which is correct for "different form of Tauros" matching but won't equal `match_key("Tauros")`. Callers doing form-aware matching should compare on the `parent_species` field, not the `cameo_subject's match_key`.

Layer 3 — `slug(name)` (filename / URL form)

Filesystem- and URL-safe. Lossy in the same ways as `match_key` plus shape-preserving hyphens.

1. `canonicalize(name)`.
2. Apply the gender symbol preprocess from Layer 2.
3. `casefold()`, then NFKD + strip combining marks (same as Layer 2 steps 3–4).
4. Drop straight apostrophes outright (so `Farfetch'd` → `farfetchd`, not `farfetch-d`).
5. Replace any run of non-`[a-z0-9]` with a single `-`.
6. Strip leading/trailing `-`.

Input	slug
Farfetch'd	farfetchd
Sirfetch'd	sirfetchd
Iron Jugulis	iron-jugulis
Mr. Mime	mr-mime
Ho-Oh	ho-oh
Type: Null	type-null
Flabébé	flabebe
Nidoran♀	nidoran-f
Tapu Koko	tapu-koko
Pokémon	pokemon
Tauros (Paldean Combat Breed)	tauros-paldean-combat-breed
Mime Jr.	mime-jr

```

1 import re
2 import unicodedata
3
4 _APOSTROPHES = {"\u2019": "'", "\u2018": '"', "\u02BC": "'"}
5 _GENDER = {"\u2640": "-f", "\u2642": "-m"}
6
7 def canonicalize(name: str) -> str:
8     s = unicodedata.normalize("NFC", name)
9     for src, dst in _APOSTROPHES.items():
10         s = s.replace(src, dst)
11     s = re.sub(r"\s+", " ", s).strip()
12     return s
13
14 def _strip_accents(s: str) -> str:
15     s = unicodedata.normalize("NFKD", s)
16     return "".join(c for c in s if not unicodedata.combining(c))
17
18 def match_key(name: str) -> str:
19     s = canonicalize(name)
20     for src, dst in _GENDER.items():
21         s = s.replace(src, dst)
22     s = _strip_accents(s.casefold())
23     return re.sub(r"^[a-z0-9]", "", s)
24
25 def slug(name: str) -> str:
26     s = canonicalize(name)
27     for src, dst in _GENDER.items():
28         s = s.replace(src, dst)
29     s = _strip_accents(s.casefold())
30     s = s.replace("'", "")
31     s = re.sub(r"^[a-z0-9]+", "-", s)
32     return s.strip("-")
33

```

7. Workflow: GitHub Actions

7.1 Trigger

Use the `released` event type. This is the one that fires when a release becomes the "Latest" pinned release on the repo sidebar — specifically, when a release is published as a non-prerelease, or when an existing pre-release is converted to a full release.

```

1 on:
2   release:
3     types: [released]
4   workflow_dispatch:      # manual re-run support
5

```

Why not `published`: `published` also fires for pre-releases, which do *not* take the "Latest" sidebar slot. Pre-releases would generate assets that misrepresent themselves as the current snapshot.

Why not `created`: `created` fires the moment a draft is saved, before publication; the release wouldn't yet be visible to consumers.

7.2 Job outline

```

1 name: Build cameo-convert release assets

```

```

2
3 on:
4   release:
5     types: [released]
6   workflow_dispatch:
7     inputs:
8       release_tag:
9         description: 'Existing release tag to attach assets to'
10        required: true
11
12  jobs:
13    build-and-attach:
14      runs-on: ubuntu-latest
15      permissions:
16        contents: write
17      steps:
18        - uses: actions/checkout@v4
19
20        - uses: actions/setup-python@v5
21          with:
22            python-version: '3.12'
23
24        - run: pip install -e .
25
26        - name: Resolve release tag
27          id: tag
28          run: |
29            if [ -n "${{ github.event.release.tag_name }}" ]; then
30              echo "value=${{ github.event.release.tag_name }}" >>
31              $GITHUB_OUTPUT
32            else
33              echo "value=${{ inputs.release_tag }}" >> $GITHUB_OUTPUT
34            fi
35
36        - name: Convert
37          run: |
38            python -m cameo_convert \
39              --input data/cameo-database.ods \
40              --output-dir dist/ \
41              --release-tag "${{ steps.tag.outputs.value }}" \
42              --bundle zip,tar.gz
43
44        - name: Upload to release
45          uses: softprops/action-gh-release@v2
46          with:
47            tag_name: "${{ steps.tag.outputs.value }}"
48            files: dist/*

```

7.3 Filename scheme

Every file gets the release tag in its name:

```

1 cameo-convert-{tag}-cards-{variant}.{ext}
2 cameo-convert-{tag}-all-formats.zip
3 cameo-convert-{tag}-all-formats.tar.gz
4

```

Examples for tag **v1.2.3** :

```

1 cameo-convert-v1.2.3-cards-clean.json
2 cameo-convert-v1.2.3-cards-clean.csv
3 cameo-convert-v1.2.3-cards-full.sqlite
4 cameo-convert-v1.2.3-cards-full.xlsx

```

```
5 cameo-convert-v1.2.3-all-formats.zip
6
```

The tag is passed through `slug()` (§6.8) defensively in case a future release uses an unusual character.

7.4 Reader implementation detail

The reader walks `content.xml` via `odfpy`:

1. Build a `style_name → properties` map from `office:automatic-styles`. Resolve parent style chains so inherited italic/background is correctly attributed.
2. For each non-`Main` sheet, walk `table:table-row → table:table-cell`. Track active vertical merges (rows-spanned countdowns per column) so `covered-table-cell` placeholders can be filled in.
3. For each row, emit one `CameoEntry` with formatting-derived flags resolved from the cell's `table:style-name`.
4. Detect shared-artwork groups by recording the row range of each column-2 merge and assigning a synthetic `artwork_group_id`.

The `Main` sheet is read once to extract the `Last Updated` date for the metadata block.

8. Testing

- **Unit:** reader on a 20-row hand-built `tests/fixtures/tiny_sample.ods` that exercises every edge case (italic, blue bg, both merge types, blank card, non-numeric #, different-form cameo, blank cells).
- **Normalization:** every example in §6.8's tables is a test case in `test_normalize.py`. Plus property-based tests asserting `canonicalize(canonicalize(x)) == canonicalize(x)` (idempotence) for all three functions.
- **Round-trip:** writers that support the full schema (JSON ↔ YAML ↔ TOML ↔ XLSX) must round-trip a fixture without loss.
- **Golden file:** full conversion of the current real `.ods` produces `dist/*.json` files diffed against checked-in goldens in CI. Goldens are regenerated when the source updates.
- **Smoke:** the GitHub workflow runs `python -m cameo_convert ... --variants clean` on every PR to catch reader regressions early.

9. Risks & Mitigations

Risk	Likelihood	Mitigation
------	------------	------------

Source <code>.ods</code> introduces a new formatting convention we don't recognise	Med	Reader logs (and CI fails on) any unknown background colour or unrecognised style combination.
LibreOffice version skew rewrites style names on every save, churning the diff	Low	Reader keys off properties, not style names.
Release workflow fires twice (e.g. release edit re-fires the trigger) and double-uploads	Low	<code>softprops/action-gh-release</code> is idempotent on file names; tag-included filenames prevent cross-release collisions.
Output file count (28) clutters the release page	Low	Bundle archives are the "front door"; individual files are for power users.
Cameo Pokémon column contains a form name that doesn't match any species name	Med	Preserve the literal in <code>cameo_subject</code> ; record block species in <code>parent_species</code> ; flag <code>DIFFERENT_FORM</code> .
Unicode handling breaks on a future Pokémon name with novel punctuation	Med	Normalization layer is centralized in <code>normalize.py</code> ; new cases need only one fix and one test.

10. Resolved Decisions

#	Decision	Resolution
---	----------	------------

D1	Card identity format	Composite <code>Set CardName #</code> string key.
D2	Reprint collapsing	No collapsing. One card entry per collector number; entries sharing artwork share an <code>artwork_group_id</code> for optional client-side grouping.
D3	Clean variant filter	English-only collector view. Excludes <code>NON_ENGLISH</code> and <code>ITALIC</code> and <code>BLANK_CARD_NAME</code> .
D4	Output formats	Adds TOML to the original proposal. Final list: JSON, JSONL, YAML, TOML, XML, CSV, TSV, MD, HTML, TXT, SQLite, XLSX, ODS (13 formats).
D5	Release trigger	<code>release:</code> <code>[released]</code> event (fires when a release becomes the "Latest" sidebar release). Filenames include the release tag.
D6	Trainers output shape	Combined with Pokémon, distinguished by <code>kind</code> discriminator in every output.

D7	Different-form cameos	Both names retained. Structured formats: one entry, both <code>subject</code> and <code>parent_species</code> . Flat formats: row emitted twice with a <code>relation</code> column.
D8	Source file path	<code>data/cameo-database.ods</code> — original filename's apostrophe and <code>é</code> removed to avoid shell quoting issues in CI.

11. Alternatives Considered

- **Use `pandas.read_excel(engine="odf")` only.** Rejected: pandas drops cell formatting, which is where italics-as-edge-case and blue-as-non-English live.
- **Convert the `.ods` to `.xlsx` first via LibreOffice in CI, then read with `openpyxl`.** Rejected: adds a heavy CI dependency (LibreOffice) and the conversion sometimes re-colours cells in subtle ways.
- **Output one big format only (JSON) and let users transform.** Rejected: the requirement is many formats; many collectors aren't programmers.
- **Run on a schedule (cron) instead of release.** Rejected: source updates land via release; coupling to release gives a single citable artifact per snapshot.
- **Collapse reprints into a single artwork entry.** Rejected per D2; matches what users physically pull.

12. Appendix

12.1 Background colors observed in the source

Hex	Frequency	Likely meaning (inferred)
<code>#cfe2f3</code>	most common	Card never released in English (documented)

#acd36c	header rows	Sheet header styling
#dcd677 , #e89483 , #9fcadf , #b7a3c3 , #9cd7c8 , #e8cc59 , #a0c6d1 , #c97dc0 , #dd608c , #7c93a9	scattered, low count	Likely region/section banners on Trainers; needs confirmation before being relied on.

The reader passes through any unrecognised colour into a `bg_color_raw` field on the entry rather than silently dropping it, so we don't lose information when conventions change.

12.2 Italic-only styles observed

23 distinct italic cell styles found, all sharing `fo:font-style="italic"` regardless of other attributes. The reader collapses these to a single `Flag.ITALIC` rather than trying to distinguish them.